Lab_Assignment_11.5.

HT NO:2303A51360

BATCH NO:29

TASK-1:

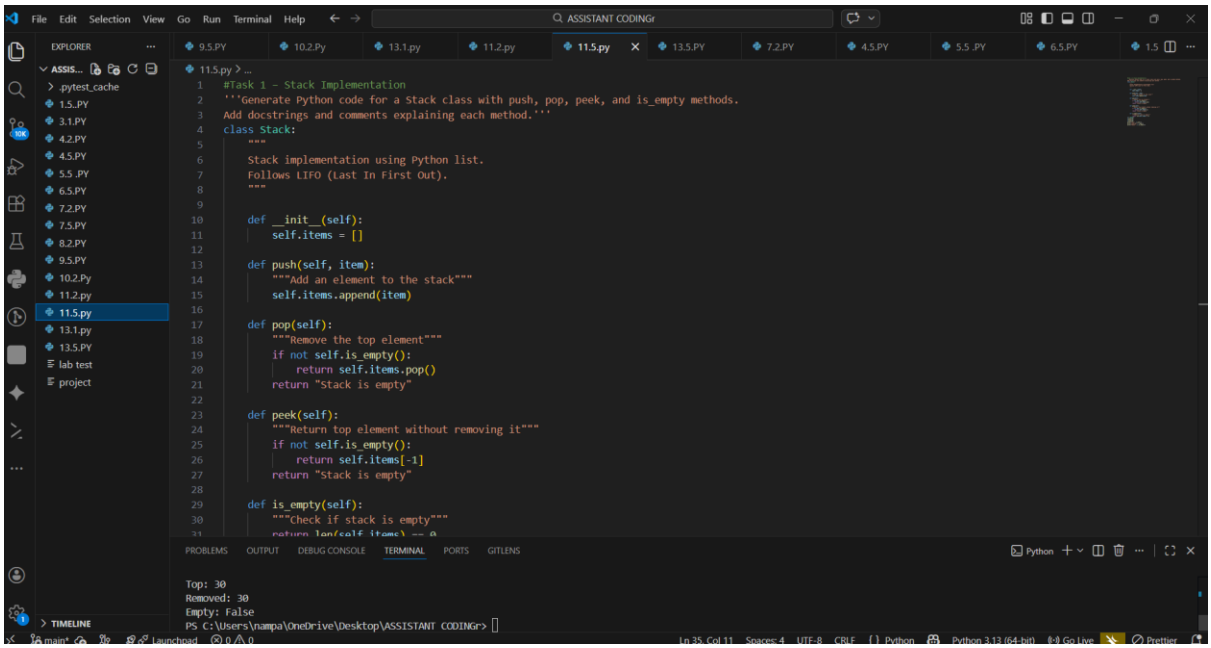
Stack Implementation

PROMPT:

Generate Python code for a Stack class with push, pop, peek, and is_emptymethods.

Add docstrings and comments explaining each method.

CODE:



```python
#Task 1 - Stack Implementation
'''Generate Python code for a Stack class with push, pop, peek, and is_empty methods.
Add docstrings and comments explaining each method.'''
class Stack:
    """
    Stack implementation using Python list.
    Follows LIFO (Last In First Out).
    """

    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an element to the stack"""
        self.items.append(item)

    def pop(self):
        """Remove the top element"""
        if not self.is_empty():
            return self.items.pop()
        return "Stack is empty"

    def peek(self):
        """Return top element without removing it"""
        if not self.is_empty():
            return self.items[-1]
        return "Stack is empty"

    def is_empty(self):
        """Check if stack is empty"""
```

```
Top: 30
Removed: 30
Empty: False
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

Stack follows **LIFO principle**.

push() inserts elements to the top.

pop() removes the most recently added element.

peek() helps view the top element without deletion.

TASK-2:

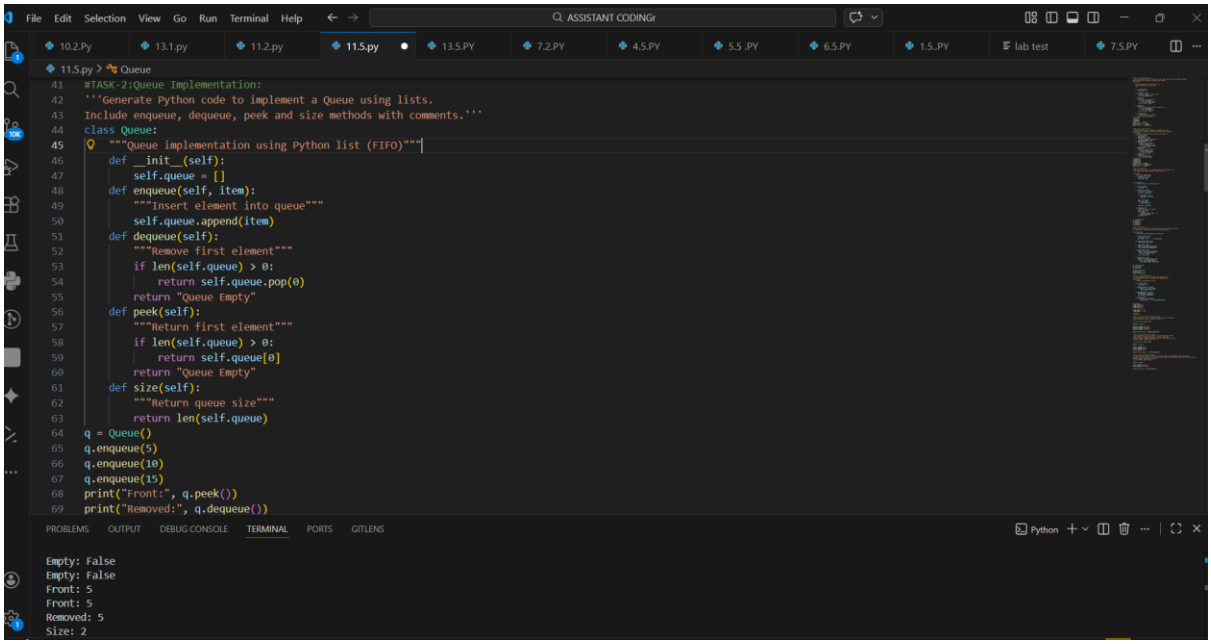
Queue Implementation

PROMPT:

Generate Python code to implement a Queue using lists.

Include enqueue, dequeue, peek and size methods with comments.

CODE:



```python
#TASK-2:Queue Implementation:
'''Generate Python code to implement a Queue using lists.
Include enqueue, dequeue, peek and size methods with comments.'''
class Queue:
    """Queue implementation using Python list (FIFO)"""
    def __init__(self):
        self.queue = []
    def enqueue(self, item):
        """Insert element into queue"""
        self.queue.append(item)
    def dequeue(self):
        """Remove first element"""
        if len(self.queue) > 0:
            return self.queue.pop(0)
        return "Queue Empty"
    def peek(self):
        """Return first element"""
        if len(self.queue) > 0:
            return self.queue[0]
        return "Queue Empty"
    def size(self):
        """Return queue size"""
        return len(self.queue)
q = Queue()
q.enqueue(5)
q.enqueue(10)
q.enqueue(15)
print("Front:", q.peek())
print("Removed:", q.dequeue())
```

```
Empty: False
Empty: False
Front: 5
Front: 5
Removed: 5
Size: 2
```

OBSERVATION:

The Queue data structure follows the **FIFO (First In First Out)** principle, where the first inserted element is removed first.

The **enqueue() method** successfully adds elements to the rear of the queue using Python lists.

The **dequeue() method** removes the element from the front of the queue, maintaining FIFO order.

The **peek() method** allows viewing the front element without removing it.

The **size() method** correctly returns the total number of elements present in the queue.

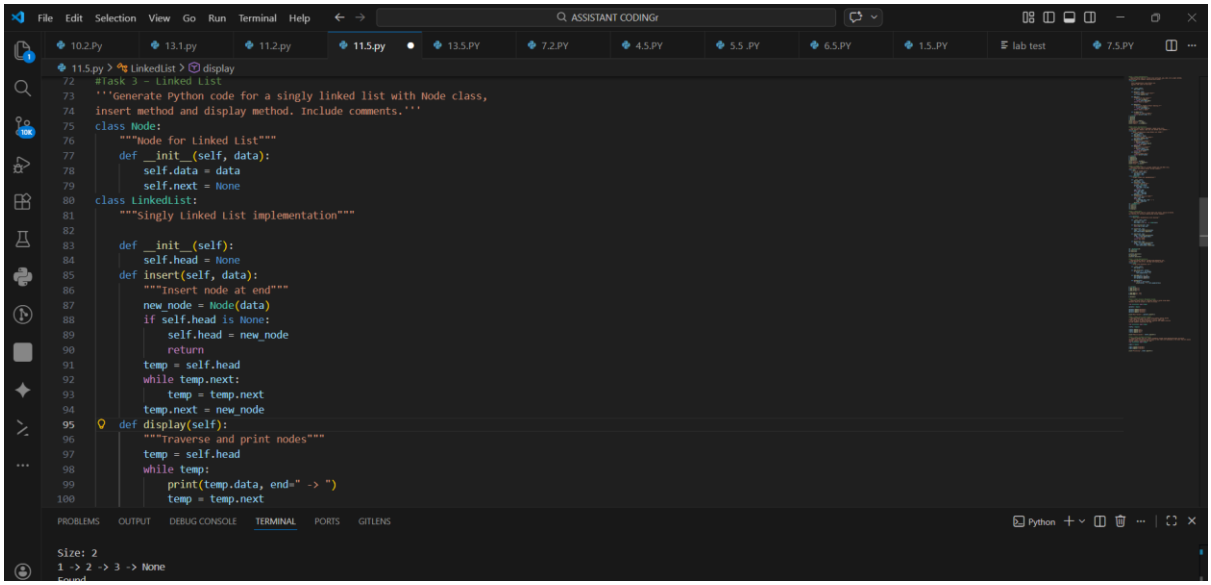
TASK-3:

Linked List

PROMPT:

Generate Python code for a Singly Linked List with Node and LinkedList classes including insert and display methods with comments.

CODE:



```python
#lask 3 - Linked List
'''Generate Python code for a singly linked list with Node class,
insert method and display method. Include comments.'''
class Node:
    """Node for Linked List"""
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    """Singly Linked List implementation"""

    def __init__(self):
        self.head = None
    def insert(self, data):
        """Insert node at end"""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    def display(self):
        """Traverse and print nodes"""
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
```

```
Size: 2
1 -> 2 -> 3 -> None
```

OBSERVATION:

The **Singly Linked List** stores data in nodes where each node contains **data and a reference to the next node**.

The **insert() method** correctly adds new nodes to the linked list.

The **display() method** traverses the list and prints all elements in order.

Linked lists allow **dynamic memory allocation**, meaning elements can be added easily without resizing.

This structure is useful when frequent **insertions and deletions** are required.

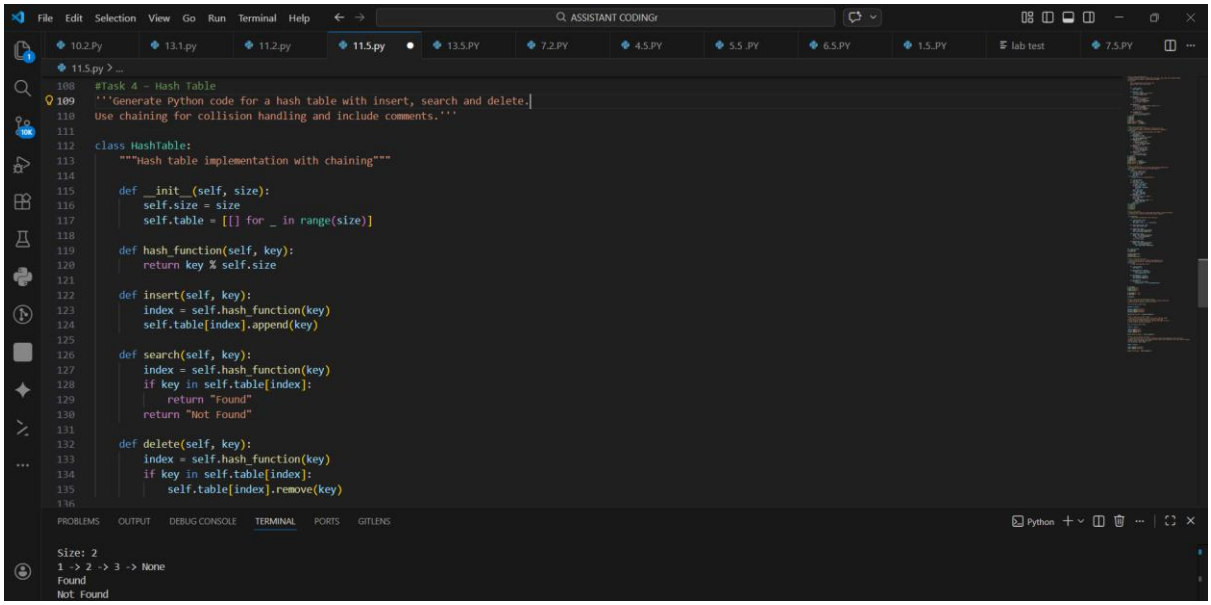
TASK-4:

Hash Table

PROMPT:

Generate Python code for a Hash Table class with insert, search, and delete methods using chaining for collision handling with comments.

CODE:



```python
#Task 4 – Hash Table
'''Generate Python code for a hash table with insert, search and delete.
Use chaining for collision handling and include comments.'''

class HashTable:
    """Hash table implementation with chaining"""

    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return key % self.size

    def insert(self, key):
        index = self.hash_function(key)
        self.table[index].append(key)

    def search(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            return "Found"
        return "Not Found"

    def delete(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            self.table[index].remove(key)
```

```
Size: 2
1 -> 2 -> 3 -> None
Found
Not Found
```

OBSERVATION:

A **Hash Table** stores data using a **hash function** to calculate an index position.

The **insert() method** places elements in the computed bucket.

Chaining helps handle collisions by storing multiple values in the same index.

The **search() and delete() methods** allow fast retrieval and removal of elements.

TASK-5:

Graph Representation

PROMPT:

Generate Python code for a Graph using an adjacency list with methods to add vertices, add edges, and display the graph with comments.

CODE:

```python
#Task 5 - Graph Representation
'''Generate Python code for a graph using adjacency list.
Include methods add_vertex, add_edge and display_graph.'''
class Graph:
    """Graph using adjacency list"""

    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, v1, v2):
        self.graph[v1].append(v2)
        self.graph[v2].append(v1)

    def display(self):
        for vertex in self.graph:
            print(vertex, "->", self.graph[vertex])
g = Graph()
g.add_vertex("A")
g.add_vertex("B")
g.add_vertex("C")

g.add_edge("A", "B")
g.add_edge("A", "C")
g.display()
```

```
Found
Not Found
A -> ['B', 'C']
B -> ['A']
C -> ['A']
```

OBSERVATION:

A **Graph** represents relationships between nodes and edges.

The **adjacency list** efficiently stores connected vertices.

The **add_vertex() and add_edge() methods** build connections between nodes.

The **display() method** shows all vertex connections clearly.

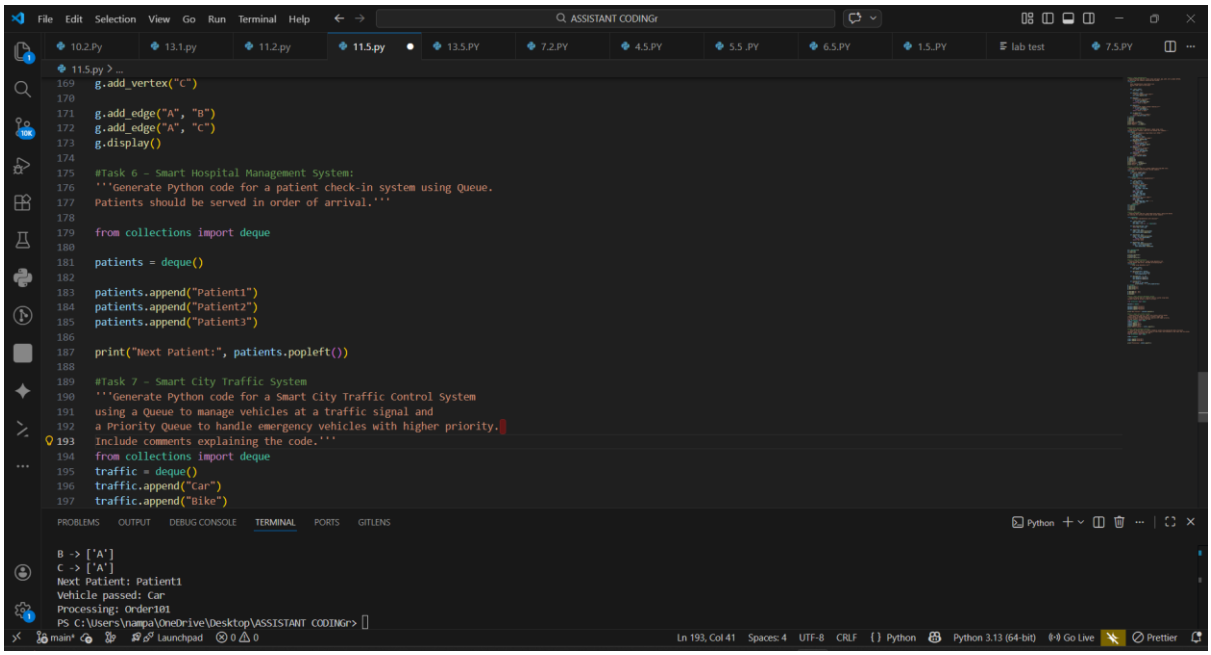
TASK-6:

Smart Hospital Management System

PROMPT:

Generate Python code for a patient check-in system using Queue where patients are treated in order of arrival with proper comments.

CODE:



```
g.add_vertex("C")

g.add_edge("A", "B")
g.add_edge("A", "C")
g.display()

#Task 6 – Smart Hospital Management System:
'''Generate Python code for a patient check-in system using Queue.
Patients should be served in order of arrival.'''

from collections import deque

patients = deque()

patients.append("Patient1")
patients.append("Patient2")
patients.append("Patient3")

print("Next Patient:", patients.popleft())

#Task 7 – Smart City Traffic System
'''Generate Python code for a Smart City Traffic Control System
using a Queue to manage vehicles at a traffic signal and
a Priority Queue to handle emergency vehicles with higher priority.
Include comments explaining the code.'''
from collections import deque
traffic = deque()
traffic.append("Car")
traffic.append("Bike")
```

```
B -> ['A']
C -> ['A']
Next Patient: Patient1
Vehicle passed: Car
Processing: Order101
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

Different hospital features require different **data structures** based on functionality.

Queue is suitable for patient check-in because patients are treated in **arrival order (FIFO)**.

Priority Queue helps manage emergency cases where critical patients are treated first.

Using proper data structures improves **efficiency and data organization** in hospital systems.

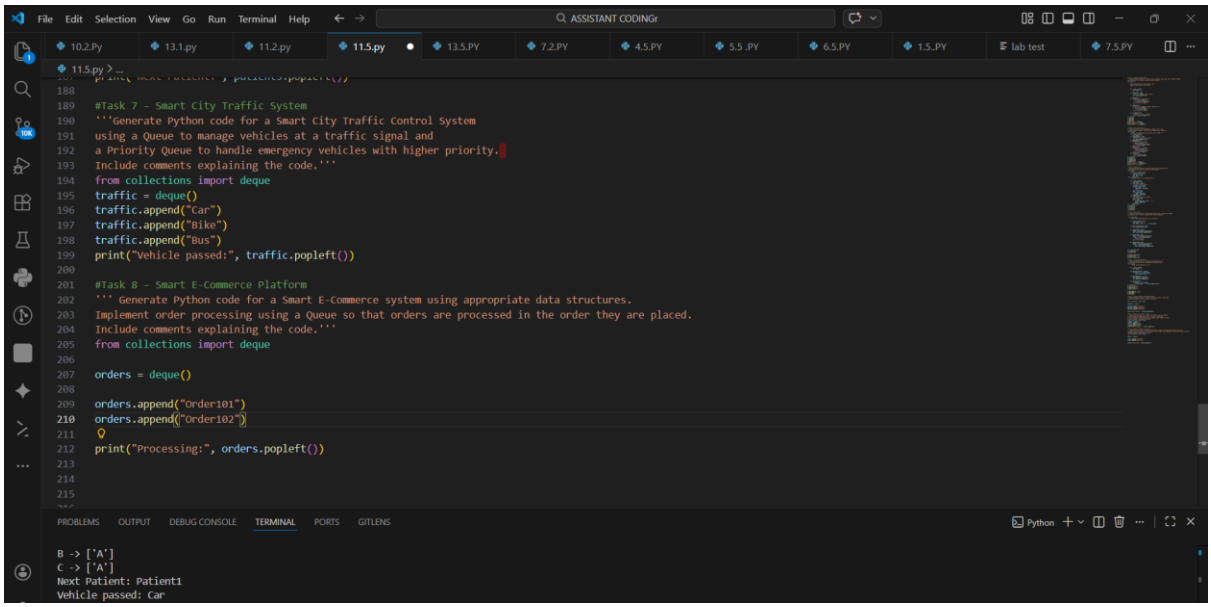
TASK-7:

Smart City Traffic Control System

PROMPT:

Generate Python code for a traffic signal management system using a Queue to process vehicles in arrival order with comments.

CODE:



```python
#Task 7 – Smart City Traffic System
'''Generate Python code for a Smart City Traffic Control System
using a Queue to manage vehicles at a traffic signal and
a Priority Queue to handle emergency vehicles with higher priority.
Include comments explaining the code.'''
from collections import deque
traffic = deque()
traffic.append("Car")
traffic.append("Bike")
traffic.append("Bus")
print("Vehicle passed:", traffic.popleft())

#Task 8 – Smart E-Commerce Platform
''' Generate Python code for a Smart E-Commerce system using appropriate data structures.
Implement order processing using a Queue so that orders are processed in the order they are placed.
Include comments explaining the code.'''
from collections import deque

orders = deque()

orders.append("Order101")
orders.append("Order102")

print("Processing:", orders.popleft())
```

```
B -> ['A']
C -> ['A']
Next Patient: Patient1
Vehicle passed: Car
```

OBSERVATION:

A **Queue** is appropriate for traffic signals because vehicles move in **FIFO order**.

Vehicles waiting at the signal are processed one by one.

Priority mechanisms can allow **emergency vehicles to move first**.

This improves **traffic flow and emergency response time**.

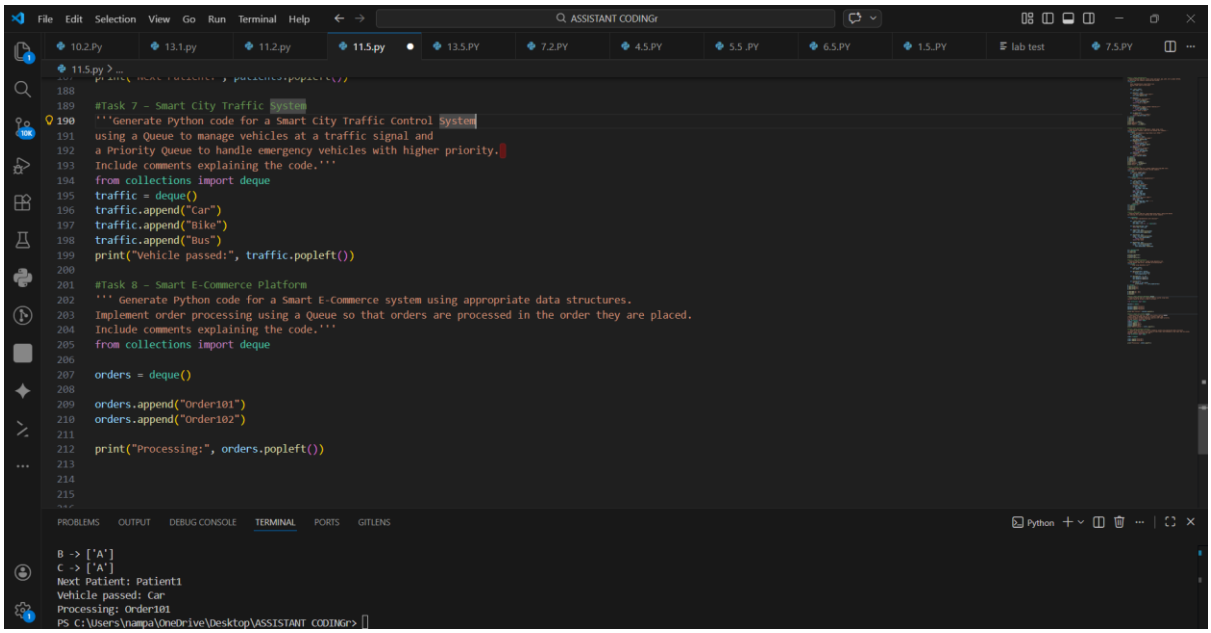
TASK-8:

Smart E-Commerce Platform

PROMPT:

Generate Python code for an order processing system using Queue so that orders are processed in the order they are placed with comments.

CODE:



```
#Task 7 – Smart City Traffic System
'''Generate Python code for a Smart City Traffic Control System
using a Queue to manage vehicles at a traffic signal and
a Priority Queue to handle emergency vehicles with higher priority.
Include comments explaining the code.'''
from collections import deque
traffic = deque()
traffic.append("Car")
traffic.append("Bike")
traffic.append("Bus")
print("Vehicle passed:", traffic.popleft())

#Task 8 – Smart E-Commerce Platform
''' Generate Python code for a Smart E-Commerce system using appropriate data structures.
Implement order processing using a Queue so that orders are processed in the order they are placed.
Include comments explaining the code.'''
from collections import deque

orders = deque()

orders.append("Order101")
orders.append("Order102")

print("Processing:", orders.popleft())
```

```
B -> ['A']
C -> ['A']
Next Patient: Patient1
Vehicle passed: Car
Processing: Order101
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

Different e-commerce features require different **data structures**.

Queue is ideal for order processing because orders must be handled in **FIFO order**.

Hash Tables help in fast product lookup using product IDs.

Data structures improve **performance, scalability, and efficient data management** in online shopping systems.